Ingeniería de Software II

Métricas

Universidad Autónoma de Entre Ríos
Facultad de Ciencia y Tecnología

Lic. Paolo Orundés Cardinali
orundescardinali.paolo@uader.edu.ar

Lic. Damián Cian
cian.damian@uader.edu.ar

Lic. Pamela Bonadeo
bonadeo.pamela@uader.edu.ar

Tabla de contenido

Definición 3

Conceptos generales 3

Aplicación de contexto 4

Ciclo de vida de las métricas 5

1. Planificación de la medición 5

2. Definición de las métricas 5

3. Recolección de datos 5

4. Análisis e interpretación 5

5. Comunicación y uso de resultados 5

6. Revisión y retroalimentación 5

Ventajas del uso de métricas 6

Mal uso de las métricas 6

1. Introducción 7

2. Actualización de conceptos tradicionales 7

1. ISO/IEC 9126 → ISO/IEC 25010 7

2. LOC/hora → Throughput, Lead Time, Story Points 7

3. Número de errores por LOC → Detección temprana y percepción de calidad 8

4. Modelo GQM → GQM+Strategies 8

5. Métricas de Complejidad → Code Smells, Code Churn, Technical Debt 8

3. Métricas modernas en desarrollo de software 8

Agilidad 8

DevOps / CI-CD 9

Calidad de Código 9

UX y Rendimiento 9

Seguridad 9

4. IA aplicada al análisis de métricas 9

5. Mini proyecto para estudiantes 9

Tipos de métricas de prueba 10

Métricas de Complejidad del Código: 10

Métricas de Calidad del Software 10

Métricas de Rendimiento 10

Métricas de Confiabilidad 11
Métricas de Productividad del Equipo 11
Métricas de Seguridad 11
Métricas de Cumplimiento 11
Métricas de Costos 12
Modelos de métricas sugeridos 13
GQM (Goal-Question-Metric) 13
GQM+Strategies 14
ISO/IEC 25010 14
CMMI (Capability Maturity Model Integration) 14
Balanced Scorecard (BSC) 14
Métricas Ágiles (Agile Metrics) 14
ITIL Metrics 15
SW-CMM (Software Capability Maturity Model) 15
Algo para jugar 16
Juego 1 16
Juego 2 17
IA aplicada al análisis de métricas 18
Conceptos clave 18
Ejemplos prácticos 18
Herramientas basadas en IA 18
Conclusión 18

Métricas

Definición

- ✔ Una métrica es una medida cuantitativa utilizada para estimar el progreso, la calidad, la productividad y el estado del proceso de prueba de software.
- ✔ Las métricas se pueden definir como “ESTÁNDARES DE MEDICIÓN”.
- ✔ Las métricas de software se utilizan para medir la calidad del proyecto. Simplemente, una métrica es una unidad utilizada para describir un atributo. La métrica es una escala para medir

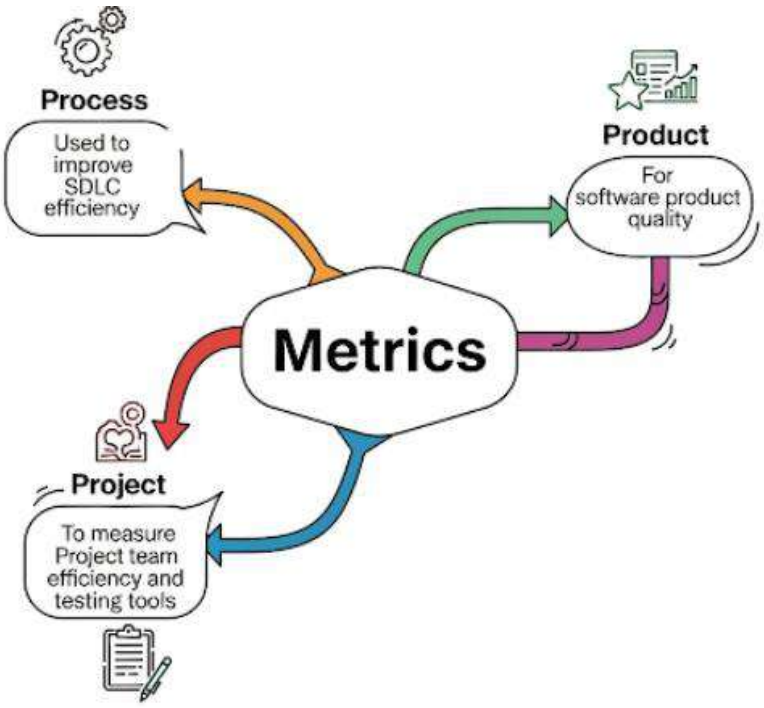
Conceptos generales

- **Atributo**: Un atributo es una característica o propiedad específica de un objeto, sistema o entidad que puede ser medible o evaluada.
    - Ejemplo: en el contexto de un producto de software, un atributo podría ser la velocidad de respuesta de una aplicación.
- **Medida**: Una medida es una cuantificación numérica de un atributo específico. Las medidas se utilizan para asignar un valor numérico a un atributo con el fin de cuantificarlo y analizarlo.
    - Ejemplo: medir la velocidad de respuesta de una aplicación en milisegundos.
- **Unidad de medida**: La unidad de medida es la unidad en la que se expresa una medida. Sirve para proporcionar contexto y significado a los valores numéricos.
    - Ejemplo: En el ejemplo anterior, la unidad de medida de la velocidad de respuesta podría ser "milisegundos".
- **Métrica**: Una métrica es una medida cuantitativa específica que se utiliza para evaluar, analizar o comparar el desempeño, la calidad o las características de un objeto o sistema.
    - Ejemplo: En el contexto del desarrollo de software, una métrica podría ser el número de errores de codificación por cada mil líneas de código.
- **Indicador**: Un indicador es una métrica específica que se utiliza para señalar una tendencia, un estado o una condición en particular. Los indicadores a menudo se utilizan para tomar decisiones o para señalar la necesidad de tomar medidas correctivas.
    - Ejemplo: un indicador de que un proyecto de desarrollo de software está en riesgo podría ser un aumento significativo en el número de errores no resueltos.



Aplicación de contexto

- **Proceso**: se puede utilizar para mejorar la eficiencia del proceso del SDLC (Ciclo de vida de desarrollo de software)
- **Producto**: Se trata de la calidad del producto de software.
- **Proyecto**: se puede usar para medir la eficiencia de un equipo de proyecto o cualquier herramienta de prueba que utilicen los miembros del equipo.



Ciclo de vida de las métricas

Basado en **Sánchez Alonso, Salvador et al. (2012),** *Ingeniería de software: un enfoque desde la guía SWEBOK*, que adapta el **SWEBOK (Software Engineering Body of Knowledge)**. El enfoque del **ciclo de vida de la medición de software** se compone de **fases iterativas**:

1. Planificación de la medición

- Se definen los **objetivos de medición** alineados con las metas del proyecto o de la organización.
- Se identifican los atributos de interés y se seleccionan las métricas más adecuadas.
    - Ejemplo: medir la calidad del código para reducir defectos en producción.

2. Definición de las métricas

- Se detallan las **métricas a emplear**, las unidades de medida, los procedimientos de recolección y los criterios de aceptación.
- Incluye documentar qué se va a medir, cómo y con qué herramientas.
    - Ejemplo: “Errores por cada 1.000 líneas de código (KLOC)”.

3. Recolección de datos

- Se ejecuta el plan: se capturan datos según los métodos definidos.
- Se asegura la **validez y consistencia** de los datos recopilados.
    - Ejemplo: usar un analizador de código (SonarQube) para contar defectos en cada revisión.

4. Análisis e interpretación

- Los datos recolectados se transforman en información útil.
- Se comparan contra **umbrales, metas o indicadores** para identificar tendencias o problemas.
    - Ejemplo: 12 errores/KLOC frente al límite de 10/KLOC → el código se considera defectuoso.

5. Comunicación y uso de resultados

- Se reportan los hallazgos a los interesados (equipo, dirección, QA).
- Se utilizan los resultados para **tomar decisiones y aplicar acciones correctivas o preventivas**.
    - Ejemplo: capacitar al equipo o refactorizar el código.

6. Revisión y retroalimentación

- Se evalúa si las métricas siguen siendo pertinentes y alineadas a los objetivos.
- Se ajustan, sustituyen o eliminan métricas para asegurar la mejora continua.
    - Ejemplo: además de errores, incorporar métricas de complejidad ciclomática o cobertura de pruebas.

En síntesis, el **SWEBOK** plantea un ciclo continuo:

Planificación → Definición → Recolección → Análisis → Comunicación → Revisión.

Ventajas del uso de métricas

- **Toma de decisiones basada en datos**: Las métricas proporcionan datos cuantitativos que permiten a los equipos de desarrollo y gestión tomar decisiones informadas. Esto puede mejorar la eficiencia y la efectividad en la toma de decisiones.
- **Control de calidad**: Las métricas ayudan a evaluar la calidad del software y permiten identificar áreas que requieren mejoras. Esto puede llevar a la corrección temprana de problemas y la entrega de un producto de mayor calidad.
- **Mejora continua**: El uso de métricas permite la retroalimentación constante y la mejora continua del proceso de desarrollo. Las métricas identifican áreas de mejora y permiten realizar ajustes en consecuencia.
- **Gestión del rendimiento**: Las métricas de rendimiento proporcionan información sobre el funcionamiento del software, lo que es esencial para optimizar su rendimiento y su eficiencia.
- **Evaluación de cumplimiento de objetivos**: Las métricas pueden ayudar a evaluar si se están cumpliendo los objetivos establecidos para un proyecto, lo que es esencial para la gestión del proyecto y la toma de decisiones estratégicas.

Mal uso de las métricas

- **Posible exceso de métricas**: La recopilación excesiva de métricas puede llevar a la sobrecarga de información y a la confusión. Es importante seleccionar cuidadosamente las métricas que sean más relevantes para el contexto y los objetivos del proyecto.
- **Manipulación incorrecta**: Si las métricas se manipulan incorrectamente o se utilizan de manera inapropiada, pueden dar lugar a decisiones erróneas o a un enfoque en la optimización de las métricas en lugar de en la mejora real del software.
- **Falta de contexto**: Las métricas por sí solas a menudo carecen de contexto. Para una interpretación adecuada, es necesario considerar el contexto en el que se recopilaron y utilizaron las métricas.
- **Enfoque en cantidad en lugar de calidad**: Centrarse exclusivamente en métricas de cantidad, como la cantidad de código escrito o las funciones implementadas, puede llevar a la producción de software de baja calidad o la ignorancia de aspectos críticos de la calidad.
- **Resistencia al cambio**: La introducción de métricas en un entorno de desarrollo puede encontrarse con resistencia por parte del equipo si se percibe como un medio de control o como una distracción de las tareas principales de desarrollo.

Actualización y Enfoques Modernos

1. Introducción

Las métricas de software permiten medir, evaluar y mejorar aspectos clave del proceso, producto y proyecto en desarrollo. En este documento se actualizan conceptos clásicos con métricas modernas, incluyendo enfoques basados en inteligencia artificial.

2. Actualización de conceptos tradicionales

A continuación, se presentan algunos conceptos tradicionales y sus versiones actualizadas:

Concepto Tradicional	Actualización / Métrica Moderna
ISO/IEC 9126	Actualizar a ISO/IEC 25010
LOC/hora	Reemplazar por Throughput, Lead Time o Story Points
Número de errores por LOC	Incluir métricas de detección temprana y percepción del usuario
Modelo GQM	Extender a GQM+Strategies para alinear con objetivos de negocio
Métricas de complejidad	Agregar Code Smells, Code Churn, Technical Debt

1. ISO/IEC 9126 → ISO/IEC 25010

- **Resumen tradicional:** ISO/IEC 9126 fue una norma que definía la calidad del software en términos de funcionalidad, confiabilidad, usabilidad, eficiencia, mantenibilidad y portabilidad.
- **Actualización:** ISO/IEC 25010 amplía y moderniza el modelo, agregando **seguridad** y **compatibilidad** entre sus 8 características de calidad. Además, diferencia entre *calidad interna*, *externa* y *en uso*.
- **Aplicación:** Evaluación formal de productos de software.
- **Normativa:** ISO/IEC 25010:2011.

2. LOC/hora → Throughput, Lead Time, Story Points

- **Resumen tradicional:** Medir productividad por la cantidad de líneas de código escritas por hora. Hoy se considera poco representativa y hasta contraproducente.
- **Actualización:**
  - **Throughput:** cantidad de historias o tareas completadas por unidad de tiempo.
  - **Lead Time:** tiempo total desde la idea hasta la entrega del producto.
  - **Story Points:** estimaciones relativas al esfuerzo que requiere una tarea.
- **Aplicación:** Equipos ágiles, Scrum, planificación orientada a valor.
- **Normativa:** No normativa formal, pero recomendado en metodologías ágiles.

3. Número de errores por LOC → Detección temprana y percepción de calidad

- **Resumen tradicional:** Se medía la calidad contando errores por cada 1000 líneas de código.
- **Actualización:** Se incorporan métricas más ricas como:
  - **Defectos detectados automáticamente (con IA)**
  - **Code Smells, debt y coverage**
  - **Encuestas de usuarios sobre calidad percibida**
- **Aplicación:** Testing automatizado, integración continua, UX.
- **Normativa:** Compatible con ISO 25010 (fiabilidad y usabilidad).

4. Modelo GQM → GQM+Strategies

- **Resumen tradicional:** GQM (Goal-Question-Metric) propone partir de un objetivo, formular preguntas y luego establecer métricas que respondan esas preguntas.
- **Actualización:** GQM+Strategies vincula los objetivos de negocio con los técnicos y operativos. Alinea estrategia organizacional y métricas del equipo de desarrollo.
- **Aplicación:** Empresas grandes, instituciones públicas, gestión de calidad estratégica.
- **Normativa:** No formal, pero ampliamente usado por NASA, Motorola, etc.

5. Métricas de Complejidad → Code Smells, Code Churn, Technical Debt

- **Resumen tradicional:** Se usaban fórmulas como la Complejidad Ciclomática para entender lo difícil que era el código.
- **Actualización:**
  - **Code Smells:** patrones de mal diseño que indican posibles errores futuros.
  - **Code Churn:** frecuencia de cambios en un módulo.
  - **Technical Debt:** acumulación de decisiones técnicas "rápidas" que comprometen la calidad a largo plazo.
- **Aplicación:** Revisión de código, mantenimiento, control de calidad interno.
- **Normativa:** Adopción generalizada en herramientas como SonarQube.

3. Métricas modernas en desarrollo de software

Las siguientes métricas reflejan prácticas actuales y herramientas modernas:

Agilidad

- Velocidad del equipo
- Lead Time
- Cycle Time
- WIP
- Sprint Goal Success Rate

DevOps / CI-CD

- Deployment Frequency
- Change Failure Rate
- MTTR
- Lead Time for Changes

Calidad de Código

- Test Coverage
- Code Smells
- Technical Debt
- Code Ownership

UX y Rendimiento

- APDEX
- Latencia
- Error Rate
- Memory/CPU Utilization

Seguridad

- Número de vulnerabilidades
- Tiempo medio de resolución
- OWASP Score

4. IA aplicada al análisis de métricas

La inteligencia artificial puede ser utilizada para:

- Analizar código automáticamente (Codex, Amazon CodeWhisperer)

- Predecir riesgos o fallas en módulos

- Explicar dashboards en lenguaje natural

- Clasificar cambios automáticamente

5. Mini proyecto para estudiantes

Actividad: Analizar un repositorio público de GitHub y generar un informe de métricas.

1. Elegir un repositorio de código abierto en GitHub (por ejemplo: Flask, Pandas o cualquier otro)
2. Identificar las siguientes métricas usando GitHub Insights o herramientas como Codacy, SonarCloud o ChatGPT:
    a. Commits por semana
    b. Número de contribuidores activos
    c. Issues abiertos/cerrados
    d. Pull Requests y tiempos de revisión

    e. Lenguajes y tamaño del código fuente
3. Utilizar ChatGPT para interpretar los datos y generar un informe en lenguaje natural.

Tipos de métricas de prueba

Métricas de Complejidad del Código:

Evalúan qué tan difícil es entender, mantener o modificar el código fuente.

- **Complejidad Ciclomática (McCabe)**: Cuenta la cantidad de caminos de ejecución independientes en el código, indicando su complejidad lógica.
- **Métricas de Halstead**: Analizan el número de operadores y operandos para estimar esfuerzo y comprensión del código.
- **Code Smells**: Patrones de código mal diseñado que indican problemas potenciales de mantenibilidad.
- **Code Churn**: Mide la frecuencia de cambios en un archivo, útil para detectar código inestable o propenso a errores.

Herramientas: SonarQube, Codacy, CodeClimate

Métricas de Calidad del Software

Miden atributos internos como legibilidad, acoplamiento y mantenibilidad.

- **Índice de Mantenibilidad (MI)**: Calcula qué tan fácil es mantener el software, basado en métricas de complejidad y volumen.
- **Índice de Cohesión y Acoplamiento**: Evalúan la dependencia interna de un módulo y su independencia respecto de otros.
- **Test Coverage**: Porcentaje del código cubierto por pruebas automatizadas.
- **Defect Density**: Cantidad de defectos por cada mil líneas de código.
- **Technical Debt Ratio**: Proporción entre el trabajo pendiente de calidad técnica y el trabajo realizado.

Herramientas: SonarQube, Coveralls, Jacoco, Pylint

Métricas de Rendimiento

Evaluación del comportamiento de una aplicación bajo carga.

- **Tiempo de respuesta promedio**: Tiempo medio que tarda el sistema en responder a una solicitud.
- **Throughput**: Cantidad de transacciones que el sistema puede manejar por segundo.
- **Uso de CPU y memoria**: Mide el consumo de recursos del sistema durante la ejecución.
- **APDEX (Application Performance Index)**: Índice que mide la satisfacción del usuario con el tiempo de respuesta de una aplicación.

Herramientas: JMeter, k6, Grafana, New Relic, Lighthouse

Métricas de Confiabilidad

Miden la capacidad del software de operar sin fallos.

- **MTBF**: Tiempo medio entre fallas; cuanto más alto, más confiable es el sistema.
- **MTTR**: Tiempo medio de recuperación tras una falla.
- **Error Rate**: Porcentaje de errores respecto al total de transacciones.
- **Success Rate en APIs**: Porcentaje de llamadas exitosas sobre el total de llamadas a una API.

Herramientas: Datadog, Prometheus, Sentry

Métricas de Productividad del Equipo

Evalúan el avance del equipo de desarrollo en términos de entregas.

- **Lead Time y Cycle Time**: Tiempo total desde la solicitud hasta la entrega (lead) y desde el inicio del trabajo hasta la entrega (cycle).
- **Historias completadas por sprint**: Cantidad de funcionalidades entregadas en cada iteración.
- **Throughput de commits**: Frecuencia con la que se hacen commits relevantes.
- **Sprint Burndown Charts**: Gráfica que muestra el trabajo restante durante un sprint.

Herramientas: Jira, Azure DevOps, GitHub Insights

Métricas de Seguridad

Evalúan vulnerabilidades y la efectividad en su resolución.

- Vulnerabilidades encontradas vs. corregidas: Relación entre los problemas detectados y los solucionados.
- Tiempo medio de remediación: Tiempo promedio que lleva corregir una vulnerabilidad.
- OWASP Top 10 mitigado: Grado de cumplimiento con las 10 amenazas más comunes según OWASP.
- Análisis estático de seguridad (SAST): Herramientas que examinan el código en busca de patrones inseguros.

Herramientas: OWASP ZAP, Snyk, Checkmarx, GitHub Dependabot

Métricas de Cumplimiento

Miden el grado de conformidad del software con normas y requisitos.

- **Cumplimiento de requisitos funcionales**: Porcentaje de funcionalidades implementadas conforme a los requerimientos.
- **Conformidad con normativas**: Verificación de cumplimiento con estándares como ISO 25010, RGPD, etc.
- **Cobertura de casos de prueba vs. requerimientos**: Grado de verificación de cada requisito con pruebas automatizadas o manuales.

Herramientas: TestRail, Zephyr, Qase

Métricas de Costos

Relacionan aspectos económicos con calidad y rendimiento.

- **Costo por defecto encontrado**: Costo promedio de detectar y corregir un defecto.
- **Costo por línea de código**: Costo asociado al desarrollo por unidad de código.
- **Costo de pruebas automatizadas vs. manuales**: Comparación de inversión y retorno entre estrategias de testing.
- **ROI de herramientas de testing**: Retorno de inversión al aplicar herramientas de pruebas automatizadas.

Herramientas: Estimaciones PM, ROI de herramientas, informes financieros

Métricas de Machine Learning

Evalúan la calidad, capacidad predictiva y comportamiento de los modelos de aprendizaje automático.

- **Accuracy (Exactitud)**: Porcentaje de predicciones correctas respecto del total de predicciones realizadas. Es útil cuando las clases se encuentran relativamente equilibradas.
- **Precision (Precisión)**: Indica qué proporción de las predicciones positivas realizadas por el modelo fueron realmente correctas.
- **Recall (Sensibilidad)**: Mide qué proporción de los casos positivos reales fueron correctamente detectados por el modelo.
- **F1-Score**: Combina Precision y Recall en una única métrica, siendo especialmente útil cuando existe desbalance entre las clases.
- **Matriz de Confusión**: Permite visualizar los verdaderos positivos, verdaderos negativos, falsos positivos y falsos negativos generados por un modelo de clasificación.
- **ROC-AUC**: Evalúa la capacidad de un modelo de clasificación para distinguir entre diferentes clases considerando distintos umbrales de decisión.
- **MAE (Mean Absolute Error)**: Mide el promedio del error absoluto entre los valores reales y los valores predichos en problemas de regresión.
- **MSE / RMSE**: Calculan el error cuadrático de las predicciones. RMSE expresa el error en la misma unidad que la variable que se intenta predecir.
- **R^2 (Coeficiente de Determinación)**: Indica qué proporción de la variabilidad de los datos puede ser explicada por un modelo de regresión.
- **Log Loss**: Evalúa la calidad de las probabilidades predichas por un modelo de clasificación, penalizando especialmente las predicciones incorrectas realizadas con alta confianza.
- **Overfitting / Underfitting**: Evalúa la diferencia entre el rendimiento del modelo con los datos de entrenamiento y con datos nuevos o de validación. Una diferencia elevada puede indicar sobreajuste.
- **Validation Score / Cross Validation**: Mide el rendimiento del modelo utilizando diferentes subconjuntos de los datos para verificar su capacidad de generalización.

Herramientas: Scikit-learn, TensorFlow, Keras, PyTorch, MLflow, Weights & Biases

Aclaración conceptual: no todas estas métricas sirven para cualquier modelo. En **clasificación** son habituales Accuracy, Precision, Recall, F1 y ROC-AUC; mientras que en **regresión** se utilizan principalmente MAE, MSE, RMSE y R^2.

Métricas Operativas de Machine Learning

Evalúan el comportamiento, eficiencia y estabilidad de los modelos de Machine Learning durante su entrenamiento y utilización en entornos reales.

- **Latencia de Inferencia**: Tiempo que tarda el modelo en generar una predicción desde que recibe los datos de entrada.
- **Throughput de Inferencia**: Cantidad de predicciones que el modelo puede procesar por unidad de tiempo.
- **Tiempo de Entrenamiento**: Tiempo requerido para entrenar o reentrenar el modelo con un conjunto de datos determinado.
- **Uso de CPU, GPU y Memoria**: Mide los recursos computacionales consumidos durante el entrenamiento y la ejecución del modelo.
- **Model Drift**: Detecta cambios en el rendimiento del modelo a lo largo del tiempo debido a modificaciones en el comportamiento del fenómeno que se intenta predecir.
- **Data Drift**: Identifica cambios en la distribución de los datos de entrada respecto de aquellos utilizados originalmente para entrenar el modelo.
- **Concept Drift**: Ocurre cuando cambia la relación entre las variables de entrada y la variable objetivo, haciendo que un modelo anteriormente válido pierda capacidad predictiva.
- **Prediction Drift**: Mide cambios significativos en la distribución de las predicciones realizadas por el modelo a lo largo del tiempo.
- **Tasa de Predicciones Inválidas o Fallidas**: Porcentaje de solicitudes que no pueden ser procesadas correctamente por el modelo.
- **Disponibilidad del Modelo**: Porcentaje de tiempo durante el cual el servicio de Machine Learning se encuentra disponible para realizar predicciones.
- **Frecuencia de Reentrenamiento**: Indica cada cuánto tiempo resulta necesario actualizar el modelo utilizando nuevos datos.
- **Costo por Inferencia**: Estima el costo computacional o económico asociado a cada predicción realizada por el modelo.

Herramientas: MLflow, Evidently AI, TensorFlow Serving, TorchServe, Prometheus, Grafana, Weights & Biases

Modelos de métricas sugeridos

GQM (Goal-Question-Metric)

- **Definición**: Modelo que establece una jerarquía lógica entre objetivos (Goal), preguntas (Question) y métricas (Metric). Fue desarrollado por Victor Basili.

- **Enfoque**: Definir objetivos claros y traducirlos en preguntas medibles, para luego diseñar métricas que permitan responderlas objetivamente.
- **Aplicación**: Evaluación de calidad de software, mejora de procesos, toma de decisiones basada en métricas.
- **Normativa**: No es una norma formal, pero ampliamente adoptado en organizaciones como NASA y Motorola.

GQM+Strategies

- **Definición**: Extensión del modelo GQM que vincula los objetivos del negocio con los objetivos técnicos y métricas.
- **Enfoque**: Alineación estratégica entre niveles organizacionales mediante objetivos y métricas relacionados.
- **Aplicación**: Gestión estratégica de métricas en organizaciones grandes o programas con múltiples áreas.
- **Normativa**: No formal, derivado del GQM clásico.

ISO/IEC 25010

- **Definición**: Estándar internacional que define características de calidad del producto software y su evaluación.
- **Enfoque**: Calidad del producto según 8 características como funcionalidad, usabilidad, seguridad, mantenibilidad, etc.
- **Aplicación**: Evaluación formal de productos software, auditorías, certificaciones de calidad.
- **Normativa**: Sí, norma ISO/IEC 25010:2011

CMMI (Capability Maturity Model Integration)

- Definición: Modelo de madurez de procesos que define niveles de mejora continua en la organización.
- **Enfoque**: Evaluación de la madurez de los procesos en 5 niveles, promoviendo la mejora progresiva.
- **Aplicación**: Gestión de procesos en desarrollo de software, ingeniería de sistemas, servicios.
- **Normativa**: Modelo adoptado por muchas organizaciones, no ISO, pero usado como base para certificaciones de calidad.

Balanced Scorecard (BSC)

- **Definición**: Modelo de gestión que utiliza indicadores en cuatro dimensiones: financiera, clientes, procesos internos, y aprendizaje.
- **Enfoque**: Medición integral del desempeño de la organización más allá de métricas financieras.
- **Aplicación**: Planificación estratégica, gestión empresarial y seguimiento de objetivos.
- **Normativa**: No es una norma ISO, pero ampliamente difundido en gestión organizacional.

Métricas Ágiles (Agile Metrics)

- **Definición**: Conjunto de métricas usadas en metodologías ágiles como Scrum o Kanban para evaluar entrega de valor y eficiencia del equipo.
- **Enfoque**: Ciclo iterativo, feedback rápido, mejora continua.

- **Aplicación**: Gestión de equipos de desarrollo ágil, planificación de sprint, análisis de productividad.
- **Normativa**: No normativa formal, pero estandarizada en guías Scrum, SAFe y Kanban.

ITIL Metrics

- **Definición**: Conjunto de métricas derivadas del marco ITIL para la gestión de servicios de TI.
- **Enfoque**: Optimización y monitoreo de servicios de TI, mejora continua en la atención y soporte.
- **Aplicación**: Service Desk, gestión de incidentes, disponibilidad de servicios, SLA.
- **Normativa**: Basado en el marco ITIL, no es norma ISO pero se alinea con ISO/IEC 20000.

SW-CMM (Software Capability Maturity Model)

- **Definición**: Modelo precursor de CMMI que evalúa la madurez de procesos de desarrollo de software.
- **Enfoque**: Organización por niveles de madurez y áreas clave de proceso.
- **Aplicación**: Evaluación de procesos de desarrollo, planificación de mejoras.
- **Normativa**: No ISO, modelo histórico utilizado hasta la adopción general de CMMI.

Algo para jugar

Como vimos anteriormente la complejidad ciclomática es una métrica que se utiliza para medir la complejidad de un programa de computadora. Fue desarrollada por Thomas J. McCabe y se utiliza para evaluar la facilidad de mantenimiento y comprensión del código fuente.

La complejidad ciclomática se basa en la estructura del flujo de control del programa y se calcula contando el número de caminos de ejecución independientes a través del código.

La métrica de complejidad ciclomática se calcula utilizando un grafo de control del programa, que se construye a partir del código fuente. Aquí está la fórmula básica para calcular la complejidad ciclomática:

$$V(G) = E - N + 2P$$

Donde:

V(G) es la complejidad ciclomática.

E es el número de aristas (transiciones) en el grafo. N es el número de nodos en el grafo.

P es el número de componentes conectados en el grafo.

Juego 1

Dada el siguiente código que define una función que verifica si un número es par o impar, calcular la complejidad ciclomática

```
def es_par_o_impar(numero):
    if numero % 2 == 0:
        print("El número es par.")
    else:
        print("El número es impar.")

numero = int(input("Ingrese un número: "))
es_par_o_impar(numero)
```

Juego 2

Dada el siguiente código que define una función que verifica si un número es primo

```python
def es_primo(numero):
    if numero <= 1:
        return False
    elif numero <= 3:
        return True
    elif numero % 2 == 0 or numero % 3 == 0:
        return False
    i = 5
    while i * i <= numero:
        if numero % i == 0 or numero % (i + 2) == 0:
            return False
        i += 6
    return True

numero = int(input("Ingrese un número: "))
resultado = es_primo(numero)

if resultado:
    print("El número es primo.")
else:
    print("El número no es primo.")
```

IA aplicada al análisis de métricas

La Inteligencia Artificial (IA) puede aplicarse al análisis de métricas de software para automatizar, optimizar y mejorar la toma de decisiones en proyectos de desarrollo. Permite detectar patrones, anticipar problemas y generar explicaciones a partir de datos numéricos.

Conceptos clave

- **Automatización**: La IA puede automatizar la recolección, interpretación y visualización de métricas.
- **Predicción**: Se pueden entrenar modelos que predicen errores, demoras o riesgos en base a métricas históricas.
- **Clasificación**: La IA puede clasificar issues, commits o módulos como defectuosos o críticos.
- **Explicabilidad**: Los modelos de lenguaje como ChatGPT permiten explicar métricas técnicas en lenguaje natural.

Ejemplos prácticos

- Un modelo de IA puede identificar archivos con alta probabilidad de contener bugs (basado en Code Churn y defect density).
- ChatGPT puede generar resúmenes automáticos de tableros de Jira o SonarQube.
- Sistemas como GitHub Copilot o Codex sugieren mejores prácticas al detectar problemas de calidad.
- Herramientas de observabilidad con IA (como Dynatrace o New Relic) detectan anomalías sin configuración previa.

Herramientas basadas en IA

- SonarQube + AI: análisis de calidad de código con alertas inteligentes.
- GitHub Copilot: asistencia de codificación con IA.
- ChatGPT: interpretación y explicación de métricas en lenguaje natural.
- Dynatrace, Datadog: detección automática de anomalías en rendimiento.

Conclusión

La IA aplicada a métricas no reemplaza al análisis humano, pero potencia la productividad, reduce el error y ayuda a interpretar grandes volúmenes de datos de forma más accesible.